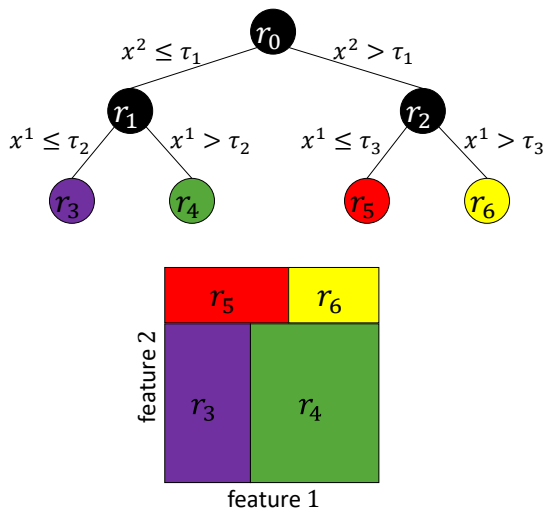


## PCS 3838 - Inteligência Artificial – Trabalho Prático 1

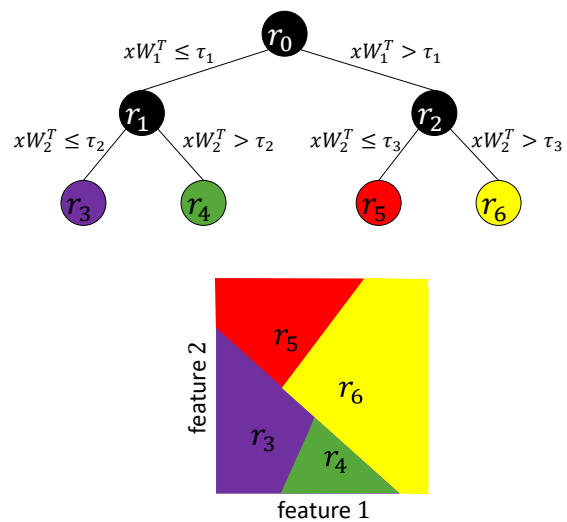
Neste trabalho você deve implementar uma forma de criar uma Floresta de Decisão **Oblíqua** (*Oblique Random Forest* – oRF). Neste classificador, as árvores que compõem a floresta são árvores de decisão oblíquas (*Oblique Decision Tree* – oDT). Diferentemente das árvores de decisão ortogonais tradicionais, onde os *splits* consideram uma única *feature* por vez, as árvores de decisão oblíquas (oDTs) particionam o espaço por meio de hiperplanos/projeções definidos por combinações, tipicamente, lineares das *features* (ex.  $xW^T$ ). A figura abaixo ilustra a diferença entre essas árvores de decisão.

Na implementação, você deve definir os hiperplanos **em cada nó da árvore**. Ou seja, **não é permitido** projetar previamente o conjunto de dados (por exemplo,  $X = XW^T$ ,  $x = xW^T$ ) e, em seguida, treinar uma árvore ortogonal. Cada nó deve aprender ou definir seu próprio hiperplano/projeção. A forma de construção dos hiperplanos é livre (por exemplo, projeções lineares aleatórias, PCA, LDA, PLS, CCA, projeção no hiperplano

Árvore de Decisão Ortogonal



Árvore de Decisão Oblíqua





do SVM, entre outros), desde que esteja claramente descrita no relatório.

A tabela abaixo sumariza as principais diferenças gerais de uma árvore ortogonal para uma oblíqua. Não é necessário seguir esta implementação exatamente.

| Componente          | Ortogonal                                                                                        | Oblíqua                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Estrutura do Node   | $i^*, \tau^*$                                                                                    | $w, \tau^*$                                                                                |
| Regra de Split      | $x^i \leq \tau$                                                                                  | $\langle \mathbf{w}, \mathbf{x} \rangle \leq \tau$                                         |
| Busca de Split      | <pre>for j in range(m):<br/>    thresholds = unique(X[:, j])<br/>    for th in thresholds:</pre> | <pre>w = ????<br/>z = X @ w<br/>thresholds = unique(z)<br/>for th in thresholds:</pre>     |
| Separação dos Dados | <pre>Xi_left = X[X[:, j] &lt;= th]<br/>Xi_right = X[X[:, j] &gt; th]</pre>                       | <pre>z = X @ w<br/>Xi_left = X[z &lt;= th]<br/>Xi_right = X[z &gt; th]</pre>               |
| Previsão            | <pre>if <math>x^{i^*} \leq \tau^*</math><br/>return prediction(x, node.left)</pre>               | <pre>z = x @ w<br/>if <math>z \leq \tau^*</math><br/>return prediction(x, node.left)</pre> |
| Tipo de Fronteira   | <i>Axis-aligned</i>                                                                              | Hiperplano oblíquo                                                                         |

Após implementar sua oRF, você deve avaliá-la na competição do Kaggle da disciplina, disponível neste link. Além disso, você deve comparar sua abordagem com uma *random forest* ortogonal (tradicional), considerando o compromisso entre habilidade preditiva (ex. acurácia) e custo computacional (ex. tempo de treinamento e/ou predição). Nesta etapa, você pode utilizar bases de dados sintéticas ou qualquer outro conjunto de dados de sua escolha. (A primeira opção permite controlar mais facilmente o tamanho dos dados para a análise).

Você deve entregar os seguintes itens (i) Relatório, (ii) Submissão no Kaggle utilizando a oRF proposta e (iii) código contendo sua implementação (um único arquivo .py) da oRF. **Não será permitida a entrega de implementações prontas de oRFs.** É permitido o uso de bibliotecas (ex. scikit-learn ou numpy) para o cálculo das projeções (ex: calcular o PCA ou treinar o SVM do nó).

Para o relatório, você deve utilizar a template disponível no e-disciplinas. O relatório deve ter no **máximo duas páginas** e conter os seguintes itens.

1. Como os hiperplanos foram definidos em cada nó da oDT (ex.: método utilizado para gerar os  $W$ s da imagem);



2. Como foram definidos os parâmetros (medida de impureza e critérios de parada de crescimento da árvore) para a criação da oDT;
3. Como foram definidos os parâmetros (número de árvores, estratégia de amostragem, estratégia para combinação das previsões, etc.) para a criação da oRF;
4. Qual foi o compromisso entre a habilidade preditiva e o custo computacional da oRF proposta comparada a uma *random forest* ortogonal (tradicional).

Para o código, entregue um arquivo `.py` que possua estrutura similar ao código abaixo.

```
1  ...
2  def main()
3      X, y = make_classification(...) #or make_moons, make_blobs...
4      X_train, X_test, y_train, y_test = train_test_split(X, y, ...)
5      model = oRF(...)
6      model.fit(X_train, y_train)
7      y_hat = model.predict(X_test)
8      accuracy_score(y_test, y_hat)
9
10 main()
```

Caso a sua implementação exija pacotes adicionais (isto é, *requirements*) para executar a oRF, detalhe no relatório quais são os pacotes e suas respectivas versões.

Qualquer violação das instruções acima pode acarretar em nota zero. Lembre-se que não será permitida a entrega de implementações prontas de oRFs.

A lista abaixo traz sugestões de leituras (opcionais) para inspirar o desenvolvimento da oRF.

1. Sarwesh Rauniar. *Jacobian Aligned Random Forests*. International Conference on Learning Representations (ICLR), 2026.



ESCOLA POLITÉCNICA DA  
UNIVERSIDADE DE SÃO PAULO

Departamento de Engenharia de  
Computação e Sistemas Digitais



- 
2. Menze et al. *On Oblique Random Forests*. ECML PKDD, 2011.
  3. O'Reilly. *Statistical Advantages of Oblique Randomized Decision Trees and Forests*. ArXiv, 2025.
  4. Nam et al. *Local Decorrelation for Improved Pedestrian Detection*. NeurIPS, 2014.